

PREDICTOR DEL GÉNERO MUSICAL DE UNA CANCIÓN USANDO ***MACHINE LEARNING***

Inteligencia Artificial y Estadística

Doble Grado Matemáticas y Estadística

Enrique Martín Luque y Javier Manzano Alcaide

DataSet: fma_small



Certificate of Completion

Dagster Essentials

This certificate is awarded to
Enrique Martín Luque

Issued: 2026-03-16

Certificate ID: sleu8hyeok

STATEMENT OF ACCOMPLISHMENT

#46,665,851

HAS BEEN AWARDED TO

Enrique Martín Luque

FOR SUCCESSFULLY COMPLETING

Writing Efficient Code with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 09, 2026




Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,689,328

HAS BEEN AWARDED TO

Enrique Martín Luque

FOR SUCCESSFULLY COMPLETING

Reshaping Data with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 16, 2026




Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,250,959

HAS BEEN AWARDED TO

Enrique Martín Luque

FOR SUCCESSFULLY COMPLETING

Python for R Users

LENGTH

5 HRS

COMPLETED ON

MAR 02, 2026




Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,610,524

HAS BEEN AWARDED TO

Enrique Martín Luque

FOR SUCCESSFULLY COMPLETING

Data Manipulation with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 09, 2026




Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,550,032

HAS BEEN AWARDED TO

Enrique Martín Luque

FOR SUCCESSFULLY COMPLETING

Joining Data with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 04, 2026




Jonathan Cornelissen
CEO, DataCamp



Certificate of Completion

Dagster Essentials

This certificate is awarded to
Javier Manzano Alcaide

Issued: 2026-04-12

Certificate ID: hwzuzz3yf9

STATEMENT OF ACCOMPLISHMENT

#46,153,949

HAS BEEN AWARDED TO

Javier Manzano Alcaide

FOR SUCCESSFULLY COMPLETING

Python for R Users

LENGTH

5 HRS

COMPLETED ON

FEB 14, 2026

 datacamp


Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,279,262

HAS BEEN AWARDED TO

Javier Manzano Alcaide

FOR SUCCESSFULLY COMPLETING

Joining Data with pandas

LENGTH

4 HRS

COMPLETED ON

FEB 19, 2026

 datacamp


Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,745,718

HAS BEEN AWARDED TO

Javier Manzano Alcaide

FOR SUCCESSFULLY COMPLETING

Writing Efficient Code with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 20, 2026

 datacamp


Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,858,917

HAS BEEN AWARDED TO

Javier Manzano Alcaide

FOR SUCCESSFULLY COMPLETING

Reshaping Data with pandas

LENGTH

4 HRS

COMPLETED ON

APR 19, 2026

 datacamp


Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,856,011

HAS BEEN AWARDED TO

Javier Manzano Alcaide

FOR SUCCESSFULLY COMPLETING

Data Manipulation with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 07, 2026

 datacamp


Jonathan Cornelissen
CEO, DataCamp

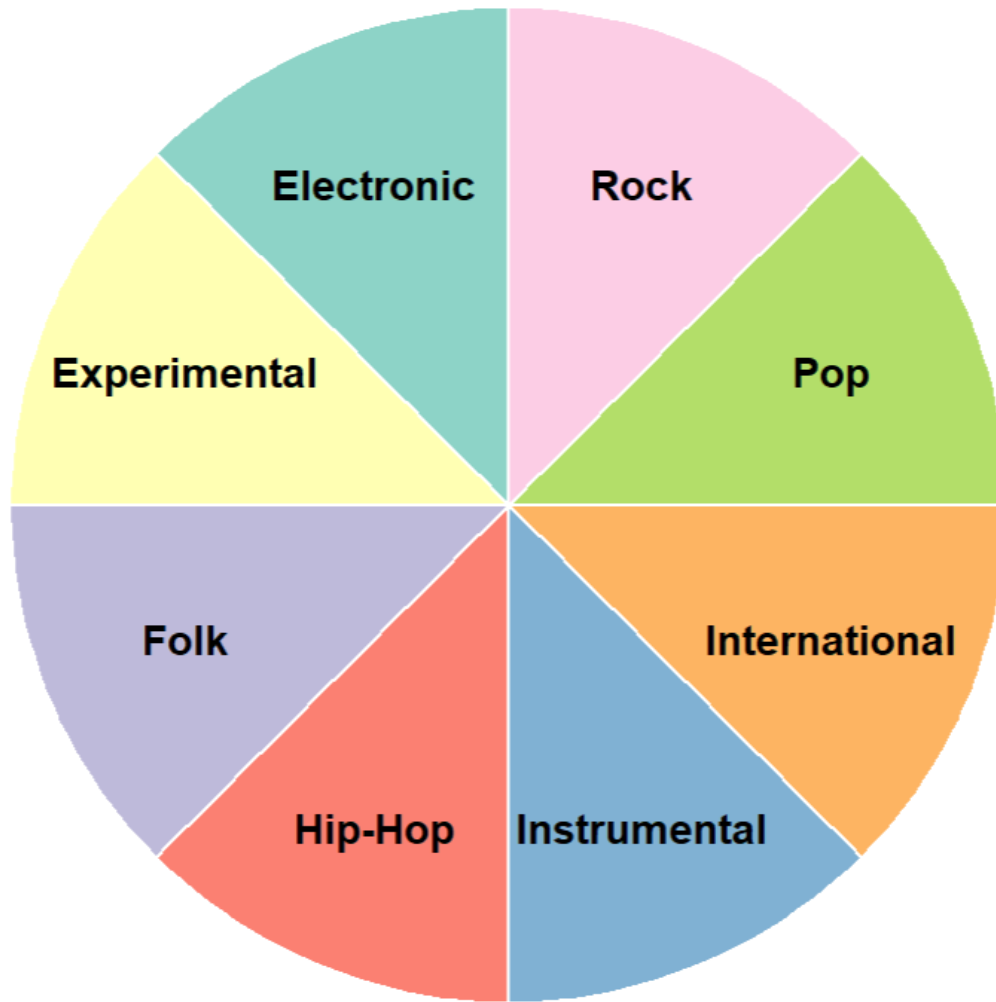
CONTEXTO DEL PROBLEMA



La música está presente en nuestra vida diaria y genera enormes cantidades de datos en plataformas digitales.

- El objetivo del trabajo es predecir el género musical de una canción a partir de sus características de audio. Para plataformas como **Spotify, YouTube Music o Apple Music**, esta información es muy valiosa, ya que permite:
 - *Organizar grandes catálogos de canciones;*
 - *Mejorar los sistemas de recomendación;*
 - *Crear playlists automáticas;*
 - *Personalizar la experiencia de cada usuario.*
- Sin embargo, el género musical no siempre está definido de forma objetiva:
 - *Mezcla de estilos.*
 - *Patrones sonoros de varios géneros.*
 - *Criterio humano.*
 - *Contexto cultural.*

BASE DE DATOS: fma_small



- Es un subconjunto del dataset **Free Music Archive** orientado a tareas de análisis musical y clasificación de género. Contiene:
 - Archivos de audio de **8000 canciones** mediante un fragmento de 30 segundos, distribuidas de forma balanceada en **8 géneros musicales**.
 - Metadatos tabulares con información técnica de las canciones.

TEMAS ABORDADOS

1. ***Introducción a Python:*** El proyecto está realizado en Python pues es uno de los lenguajes más adecuados para este tipo de proyectos, ya que permite integrar en un mismo flujo todo el proceso: carga y preprocesamiento del audio, extracción de características, entrenamiento de modelos, evaluación y despliegue de la aplicación web.
2. ***Importación de Datos:*** Hemos obtenido los archivos de audio y las tablas de metadatos de Free Music Archive mediante herramientas de Python.
3. ***Transformaciones:*** Hemos transformado los datos para facilitar el resto de tareas y obtener la variable objetivo.
4. ***Orquestadores:*** Hemos usado Dagster para entrenar el modelo XGBoost y evaluar su rendimiento. El orquestador ayuda a gestionar ese pipeline de forma más estructurada y evita depender de ejecutar scripts manualmente en un orden concreto.
5. ***Mapeo:*** Para aplicar una misma función o transformación a muchos datos de forma automática.

TEMAS ABORDADOS

1. **Ordenación:** Para facilitar la visualización y comprensión de los datos, los ordenamos según varios criterios, como el número de reproducciones de cada canción.
2. **Visualización:** Construimos diversos gráficos para ayudar a la comprensión del proyecto y la base de datos.
3. **Comunicación:** Presentación de resultados en la aplicación web realizada con streamlit.
4. **BigData:** Uso del paquete Dask de Python para acelerar el procedimiento y código eficiente para trabajar con archivos grandes.
5. **Modelización:** El proyecto consiste en un problema de clasificación de una canción según la variable objetivo “género musical” de la base de datos fma_small, para ello, desarrollamos diversos modelos de Machine Learning, Deep Learning y Transfer Learning.

IMPORTACIÓN DE DATOS



```
import pandas as pd

tracks = pd.read_csv("/content/drive/MyDrive/ProyectoIA/data/fma_metadata/tracks.csv", header=[0,1], index_col=0)
echonest = pd.read_csv("/content/drive/MyDrive/ProyectoIA/data/fma_metadata/echonest.csv", header=[0,1,2], index_col=0)
genres = pd.read_csv("/content/drive/MyDrive/ProyectoIA/data/fma_metadata/genres.csv")
features = pd.read_csv("/content/drive/MyDrive/ProyectoIA/data/fma_metadata/features.csv", header = [0,1,2], index_col = 0)
```

- **Tracks:** Contiene información de las canciones, como el autor, la fecha de publicación... de aquí obtendremos la etiqueta del género musical.
- **Echonest:** Contiene información de las canciones como el ritmo, timbre, bailabilidad... no obstante no contiene información de todas las canciones.
- **Genres:** Da información de los 8 géneros disponibles.
- **Features:** Contiene las características acústicas precomputadas de las canciones. En vez de utilizar directamente el audio bruto, este archivo permite trabajar con una representación numérica de cada pista, formada por variables como MFCC, chroma, contraste espectral o zero crossing rate. Estas variables son las que después utilizaremos en los modelos tabulares para aprender patrones asociados a cada género musical.

LIMPIEZA, ORDENACIÓN Y TRANSFORMACIÓN DE DATOS

Como los datos están disponibles para muchas más canciones, nos quedamos con las correspondientes al subset small, que es con el que estamos trabajando.

```
tracks_small = tracks[tracks[("set", "subset")] == "small"]
```

```
features_small = features.loc[ids_small]
```

```
common_ids = echonest.index.intersection(ids_small)  
echonest_small = echonest.loc[common_ids]
```

Ordenamos tracks

```
tracks_small = tracks_small.sort_values(by=('track', 'listens'), ascending=False)
```

Obtenemos la variable objetivo

```
objetivo = tracks_small[["track", "genre_top"]]
```

LIMPIEZA, ORDENACIÓN Y TRANSFORMACIÓN DE DATOS

```
import pandas as pd

# Seleccionamos las columnas más importantes de tracks_small
columns_to_keep = [
    ('track', 'title'),
    ('album', 'title'),
    ('artist', 'name'),
    ('track', 'genre_top'),
    ('track', 'duration'),
    ('track', 'listens'),
    ('set', 'split'),
    ('set', 'subset')
]

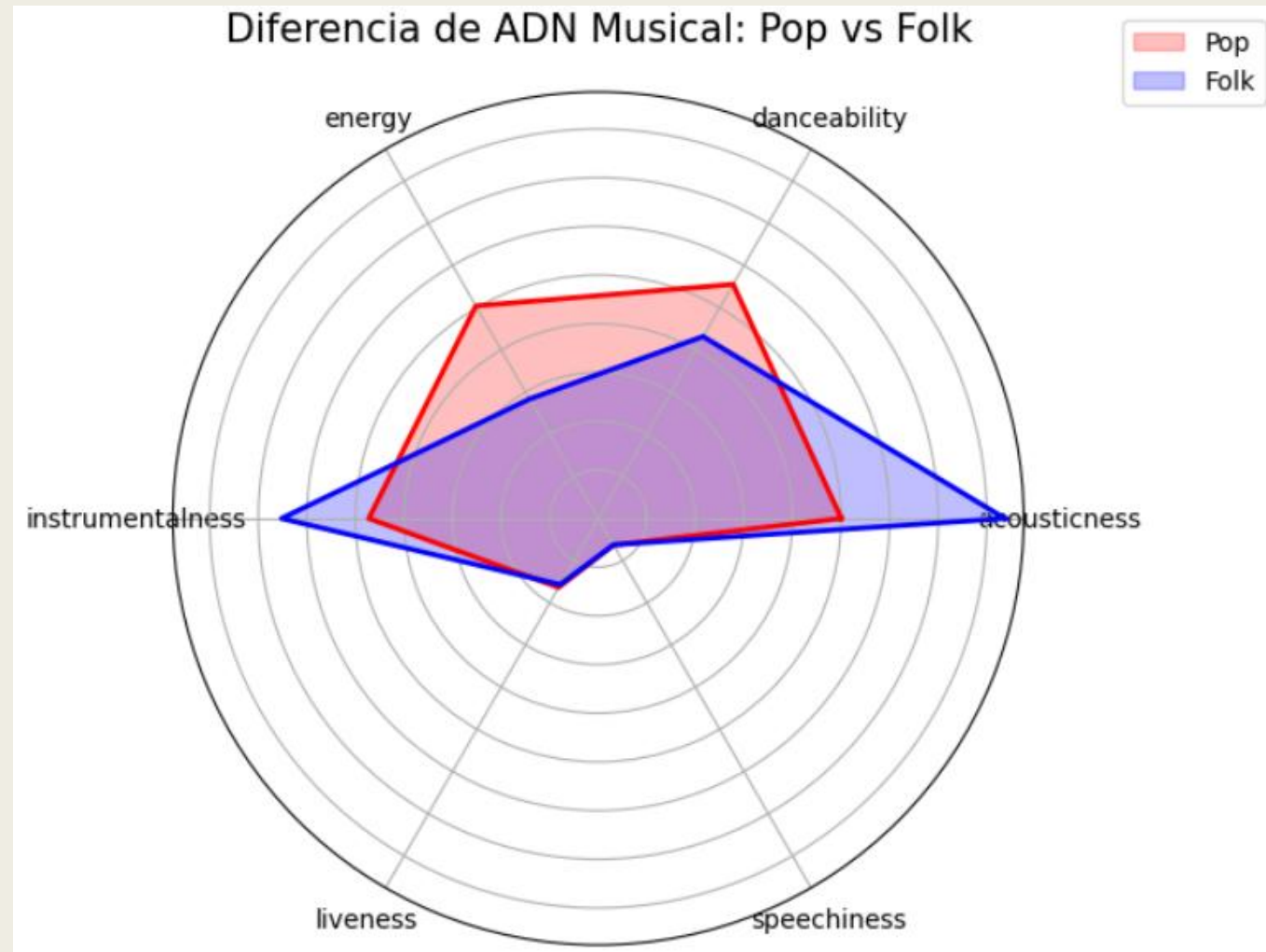
# Aplicar el filtro de columnas

tracks_small_clean = tracks_small[columns_to_keep].copy()

tracks_small_clean.columns = ['_'.join(col).strip() for col in tracks_small_clean.columns]
```

Nos quedamos solo con las columnas con las que vamos a trabajar

VISUALIZACIÓN DE DATOS



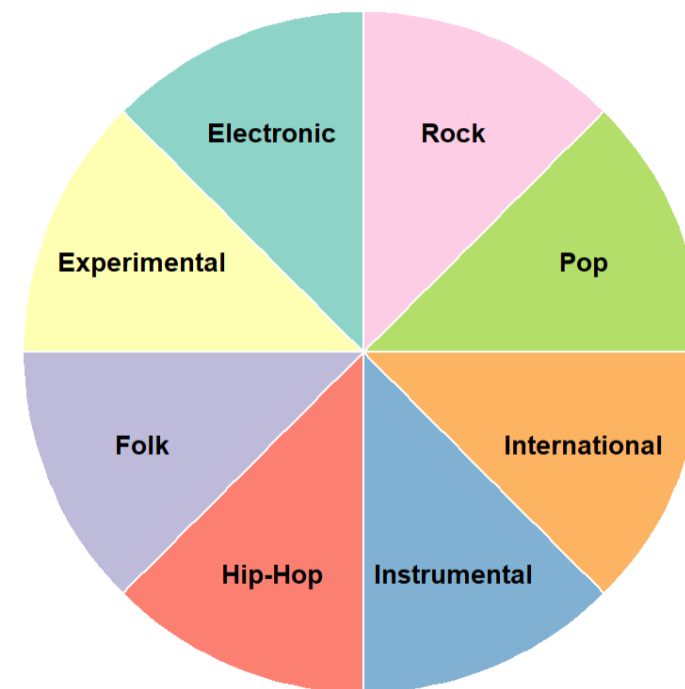
VISUALIZACIÓN (Usando R)

```
ggplot(data_plot, aes(x = "", y = count, fill = genre_top)) +  
  geom_bar(stat = "identity", width = 1, color = "white") +  
  coord_polar("y", start = 0) +  
  theme_void() +  
  labs(title = "Géneros Musicales Disponibles",  
        subtitle = "¿A cuál pertenece tu canción?") +  
  # Ajustamos 'x' para alejar el texto del centro  
  # 1 es el centro de la porción, >1 se mueve hacia afuera  
  geom_text(aes(x = 1.2, label = genre_top),  
            position = position_stack(vjust = 0.5),  
            color = "black",  
            fontface = "bold",  
            size = 4) +  
  scale_fill_brewer(palette = "Set3") +  
  theme(legend.position = "none")
```

Usamos R por su facilidad y simplicidad para realizar gráficos, que llevamos trabajando durante toda nuestra formación.

Géneros Musicales Disponibles

¿A cuál pertenece tu canción?



OBTENCIÓN DE LOS METADATOS

```
def calcular_estadisticas(caracteristica_matriz, nombre_caracteristica):  
    """Calcula las 7 estadísticas del FMA para una matriz de audio de librosa."""  
    estadisticas = {  
        'mean': np.mean(caracteristica_matriz, axis=1),  
        'std': np.std(caracteristica_matriz, axis=1),  
        'skew': stats.skew(caracteristica_matriz, axis=1),  
        'kurtosis': stats.kurtosis(caracteristica_matriz, axis=1),  
        'median': np.median(caracteristica_matriz, axis=1),  
        'min': np.min(caracteristica_matriz, axis=1),  
        'max': np.max(caracteristica_matriz, axis=1)  
    }  
  
    features_aplanadas = {}  
    for nombre_est, valores in estadisticas.items():  
        for i, valor in enumerate(valores):  
            num_str = f"{i+1:02d}"  
            col_name = f"{nombre_caracteristica}_{nombre_est}_{num_str}"  
            features_aplanadas[col_name] = valor  
  
    return features_aplanadas
```

Definimos una función para obtener los metadatos correspondientes a features.csv de una canción nueva, para poder aplicar posteriormente los modelos tabulares y clasificarla en una de las categorías.

Para ello hacemos uso de la librería librosa de Python.

OBTENCIÓN DEL ESPECTOGRAMA

```
@st.cache_data(show_spinner="🌀 Dibujando el espectrograma para la Red Neuronal...")
def procesar_audio_para_dl(uploaded_file):
    """Genera la imagen del espectrograma para la CNN."""
    with tempfile.NamedTemporaryFile(delete=False, suffix=".mp3") as tmp_file:
        tmp_file.write(uploaded_file.getvalue())
        ruta_temporal = tmp_file.name

    try:
        y, sr = librosa.load(ruta_temporal, sr=22050, mono=True, duration=30.0)
        S = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
        S_db = librosa.power_to_db(S, ref=np.max)

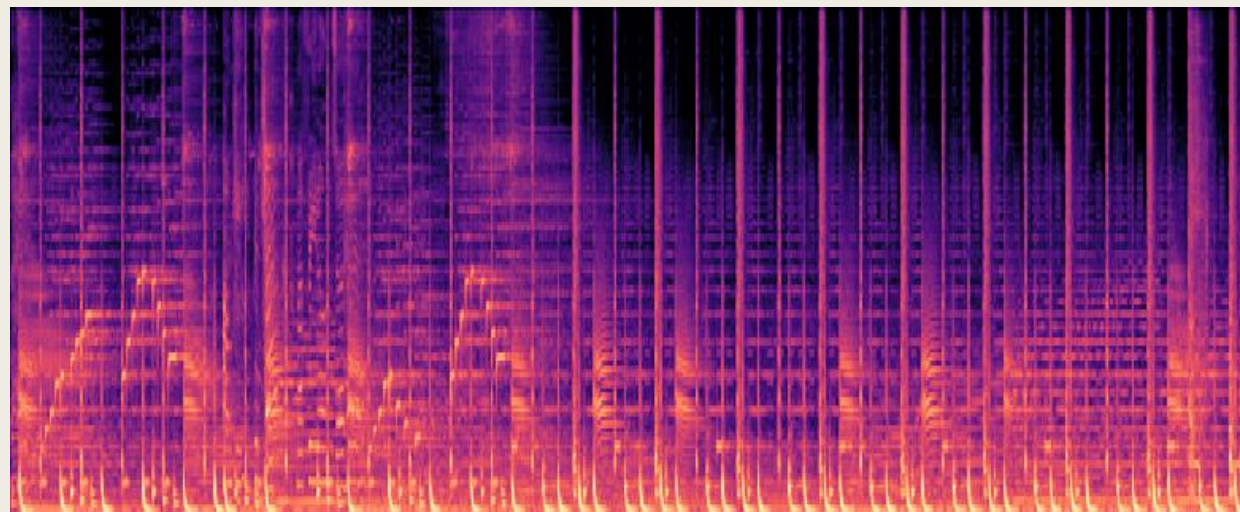
        fig = plt.figure(figsize=(2.56, 1.28), dpi=100)
        ax = plt.Axes(fig, [0., 0., 1., 1.])
        ax.set_axis_off()
        fig.add_axes(ax)

        librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='mel', ax=ax)

        buf = io.BytesIO()
        plt.savefig(buf, format='png')
        plt.close(fig)
        buf.seek(0)

        img = Image.open(buf).convert('RGB')
        img = img.resize((256, 128))
        img_array = np.array(img)
        audio_input = np.expand_dims(img_array, axis=0)
```

Haciendo uso de librosa, definimos una función para obtener un espectrograma de las canciones, que después usaremos para el modelo de Deep Learning, basado en redes neuronales artificiales



ENTRENAMIENTO DE MODELOS:

Machine Learning Clásico Tabular:

XGBoost

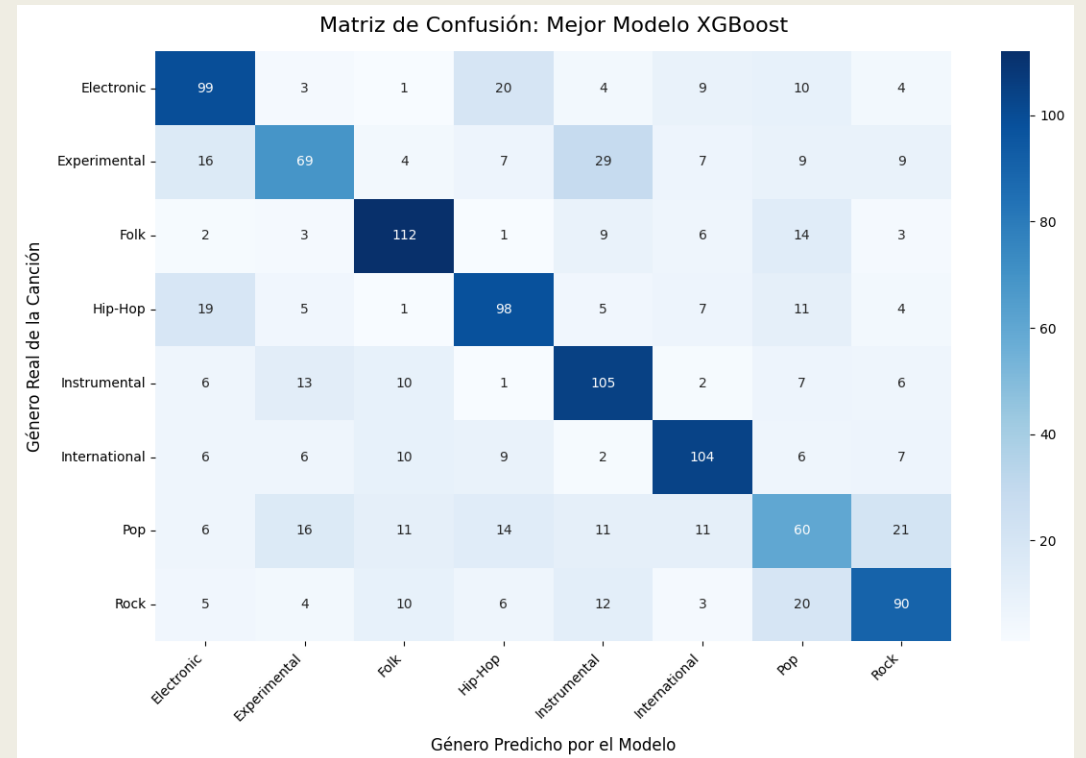
```
# 1. Configuramos el modelo con la combinación actual
modelo_prueba = xgb.XGBClassifier(
    n_estimators=150,
    max_depth=profundidad,
    learning_rate=lr,
    subsample=0.8,
    colsample_bytree=0.5,
    random_state=42,
    n_jobs=-1,
    tree_method='hist'
)

# 2. Entrenamos en el set de Entrenamiento
modelo_prueba.fit(X_train, y_train)

# 3. EVALUAMOS en el set de Validación (¡Nunca en el de Test!)
y_pred_val = modelo_prueba.predict(X_val)
precision_val = accuracy_score(y_val, y_pred_val)

print(f"Probando max_depth={profundidad}, learning_rate={lr} -> Precisión Val: {precision_val:.4f}")

# 4. Guardamos el mejor modelo
if precision_val > mejor_precision:
    mejor_precision = precision_val
    mejores_parametros = {'max_depth': profundidad, 'learning_rate': lr}
    mejor_modelo = modelo_prueba
```



ENTRENAMIENTO DE MODELOS:

Machine Learning Clásico Tabular:

SVM (Vectores Soporte)

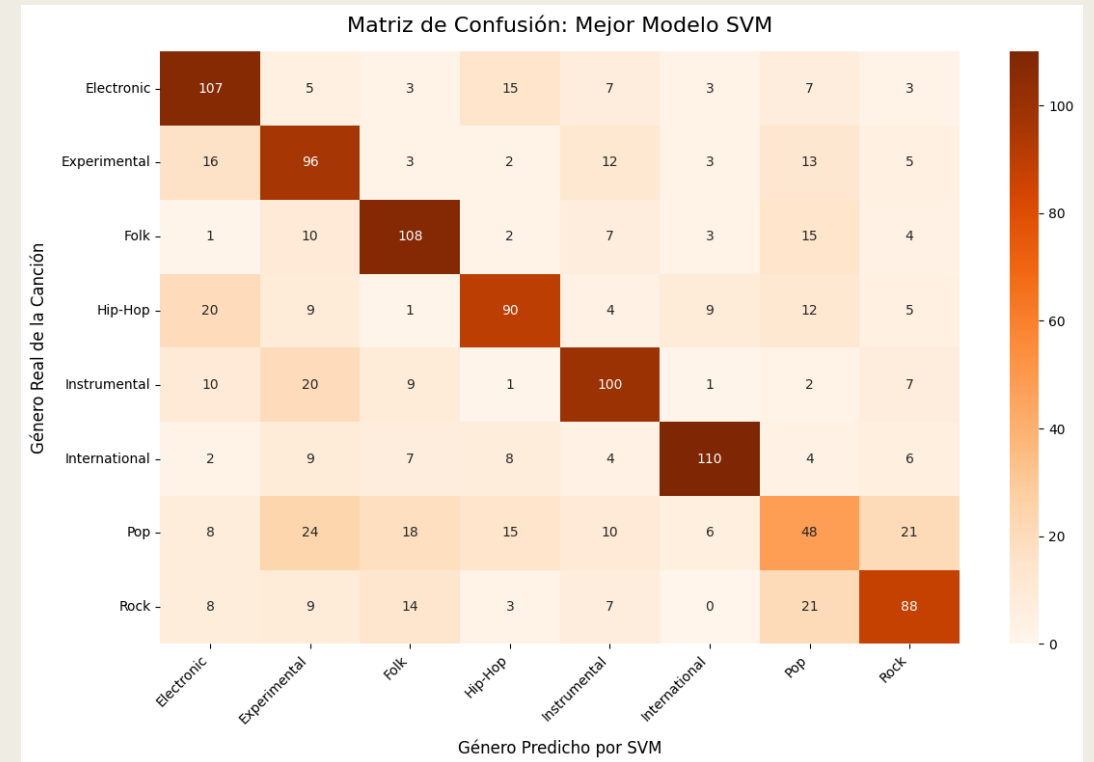
```
# 1. Configuramos el modelo con la combinación actual (Kernel RBF por defecto)
modelo_prueba_svm = SVC(
    kernel='rbf',
    C=valor_c,
    gamma=valor_gamma,
    random_state=42
)

# 2. Entrenamos en el set de ENTRENAMIENTO escalado
modelo_prueba_svm.fit(X_train_scaled, y_train)

# 3. Evaluamos en el set de VALIDACIÓN escalado
y_pred_val = modelo_prueba_svm.predict(X_val_scaled)
precision_val = accuracy_score(y_val, y_pred_val)

print(f"Probando C={valor_c}<4}, gamma={str(valor_gamma):<5} -> Precisión Val: {precision_val:.4f}")

# 4. Guardamos el mejor modelo
if precision_val > mejor_precision_svm:
    mejor_precision_svm = precision_val
    mejores_parametros_svm = {'C': valor_c, 'gamma': valor_gamma}
    mejor_modelo_svm = modelo_prueba_svm
```



ENTRENAMIENTO DE MODELOS:

Machine Learning Clásico Tabular:

Regresión Logística

```
for c in opciones_C:
    for solver in opciones_solver:

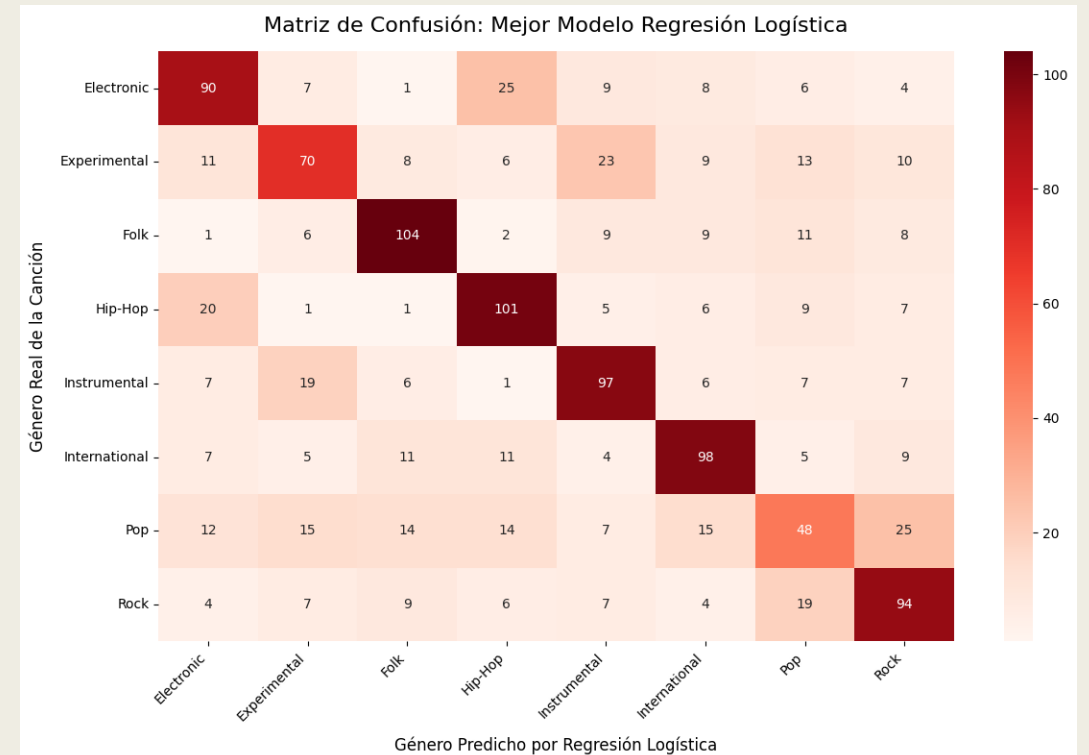
        # 1. Configuramos el modelo
        modelo_prueba_lr = LogisticRegression(
            C=c,
            solver=solver,
            max_iter=1000,
            random_state=42,
            n_jobs=-1
        )

        # 2. Entrenamos
        modelo_prueba_lr.fit(X_train_scaled, y_train)

        # 3. Evaluamos en validación
        y_pred_val = modelo_prueba_lr.predict(X_val_scaled)
        precision_val = accuracy_score(y_val, y_pred_val)

        print(f"Probando C={c:<5}, solver={solver:<7} -> Precisión Val: {precision_val:.4f}")

        # 4. Guardamos el mejor modelo
        if precision_val > mejor_precision_lr:
            mejor_precision_lr = precision_val
            mejores_parametros_lr = {'C': c, 'solver': solver}
        mejor_modelo_lr = modelo_prueba_lr
```



ENTRENAMIENTO DE MODELOS: Deep Learning usando Espectogramas

```
from tensorflow.keras import layers, models

print("🍷 Construyendo la Red Neuronal Convocucional...")

num_clases = len(nombres_clases)

modelo_cnn = models.Sequential([
    # 1. PREPARACIÓN: Convertimos los colores (0-255) a valores entre 0 y 1
    layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),

    # 2. EXTRACCIÓN DE CARACTERÍSTICAS (Las Convoluciones)
    # Capa 1: Busca patrones súper básicos
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(), # Reduce la imagen a la mitad quedándose con lo más fuerte

    # Capa 2: Busca patrones más complejos (acordes, ritmos)
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),

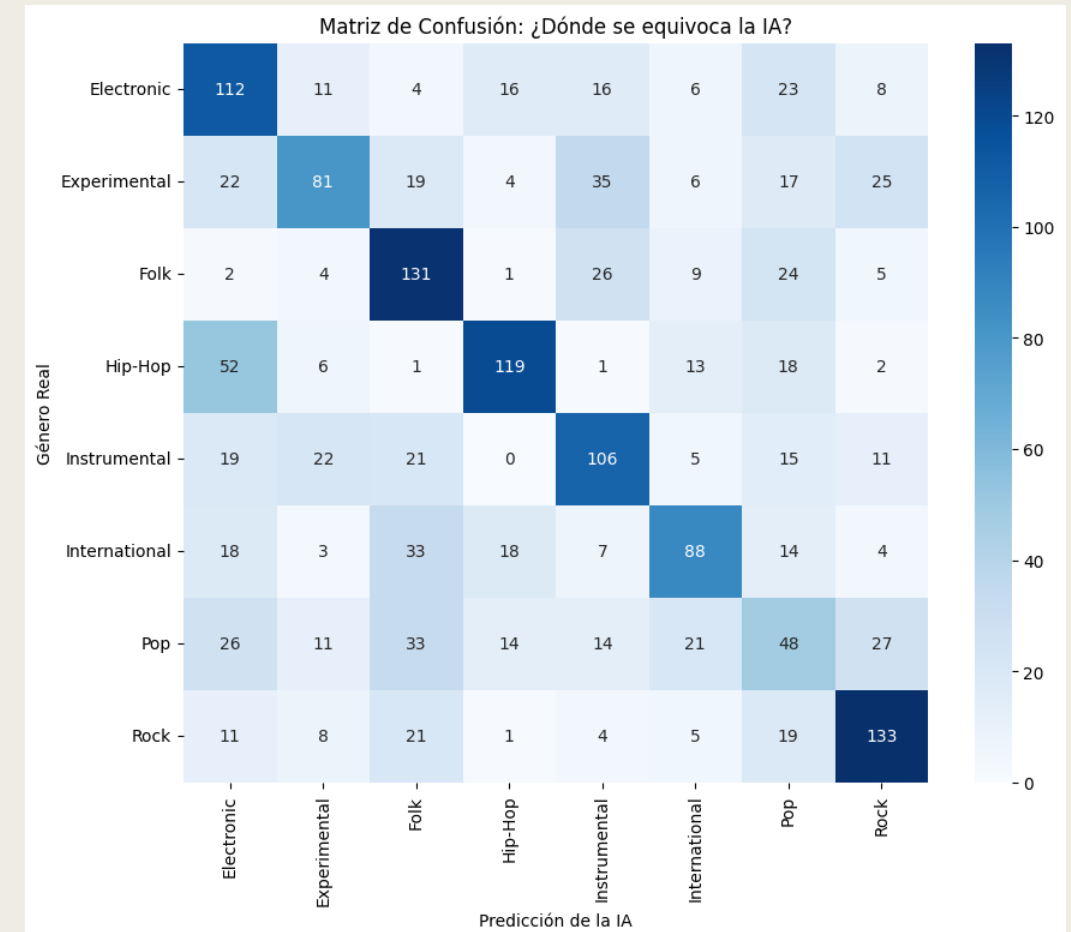
    # Capa 3: Busca conceptos de alto nivel
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),

    # 3. TOMA DE DECISIONES
    # Aplanamos la imagen 2D a una fila de números
    layers.Flatten(),

    # Neuronas clásicas que piensan sobre lo que han visto las capas anteriores
    layers.Dense(128, activation='relu'),

    # IMPORTANTE: El Dropout apaga neuronas al azar para evitar que la red
    # se "memorice" las canciones de memoria (Evita el Overfitting)
    layers.Dropout(0.5),

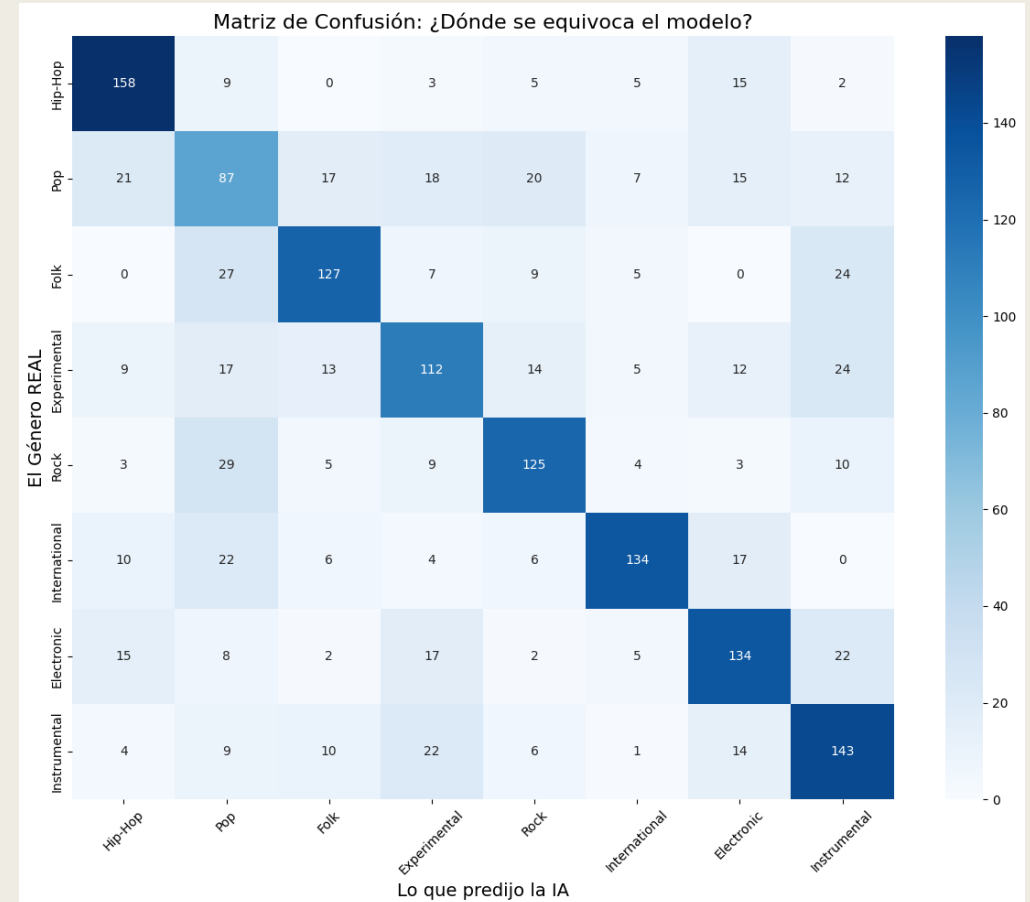
    # Capa final de salida: 8 neuronas, una por cada género
    layers.Dense(num_clases, activation='softmax')
])
```



ENTRENAMIENTO DE MODELOS:

Transfer Learning usando Yamnet

```
def extraer_embedding(ruta_mp3):  
    # YAMNet exige estrictamente audio a 16000 Hz, mono  
    y, sr = librosa.load(ruta_mp3, sr=16000, mono=True, duration=30)  
  
    # Convertir a tensor de TensorFlow  
    waveform = tf.convert_to_tensor(y, dtype=tf.float32)  
  
    # YAMNet devuelve 3 cosas: predicciones, embeddings y el espectrograma  
    _, embeddings, _ = yamnet_model(waveform)  
  
    # YAMNet da un embedding por cada medio segundo.  
    # Calculamos la media de todos para tener 1 solo vector por canción  
    embedding_promedio = tf.reduce_mean(embeddings, axis=0).numpy()  
    return embedding_promedio
```



MAPEO Y BIG DATA (DASK)

```
@asset
def preparar_datos_fma():
    """PASO 1: Carga y divide los datos usando Dask."""

    X_dask = dd.read_csv("features_small.csv", header=None)
    y_dask = dd.read_csv("objetivo.csv", header=None)

    X = X_dask.compute()
    df_etiquetas = y_dask.compute()

    # Aplicamos la función de orden superior que hicimos antes
    y = df_etiquetas.iloc[:, 0].apply(lambda x: str(x).strip().capitalize())
    y.index = X.index

    # Codificamos y guardamos los nombres reales para luego
    le = LabelEncoder()
    y_num = le.fit_transform(y)
    nombres_clases = le.classes_.tolist()

    X_train, X_test, y_train, y_test = train_test_split(
        X, y_num, test_size=0.2, random_state=42, stratify=y_num
    )

    return {
        "X_train": X_train, "X_test": X_test,
        "y_train": y_train, "y_test": y_test,
        "nombres_clases": nombres_clases
    }
```

```
@asset
def explicabilidad_modelo(entrenar_xgboost):
    """PASO 3B: Extrae el Top 5 de variables más importantes."""

    modelo = entrenar_xgboost

    # XGBoost guarda la importancia de cada variable
    importancias = modelo.feature_importances_

    df_imp = pd.DataFrame({
        "Variable": [f"Feature_{i}" for i in range(len(importancias))],
        "Importancia": importancias
    })

    df_imp["Importancia"] = df_imp["Importancia"].apply(lambda x: round(x, 4))

    df_imp = df_imp.sort_values(by="Importancia", ascending=False)

    top_5 = df_imp.head(5).to_markdown(index=False)

    return Output(
        value="Explicabilidad extraída",
        metadata={
            "Top_5_Variables": MetadataValue.md(f"### Variables que más deciden el género\n{top_5}")
        }
    )
```

ORQUESTADORES



ORQUESTADORES

```
modelo_base = xgb.XGBClassifier(random_state=42)

buscador = RandomizedSearchCV(
    estimator=modelo_base,
    param_distributions=espacio_parametros,
    n_iter=10,
    scoring='accuracy',
    cv=3,
    verbose=1,
    random_state=42
)

cliente = Client(processes=False)

with joblib.parallel_backend('dask'):
    buscador.fit(X_train, y_train)
```

```
cliente = Client(processes=False)

with joblib.parallel_backend('dask'):
    buscador.fit(X_train, y_train)

# Apagamos el clúster para liberar memoria
cliente.close()
```

```
espacio_parametros = {
    'n_estimators': [100, 200, 300],
    'max_depth': [4, 6, 8, 10],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.5, 0.7, 1.0]
}
```


COMUNICACIÓN

<https://machine-learning-genre-classification-odfyakyz9rrp3qt3dpvnue.streamlit.app/>



FMA Classifier

FMA Small · 8 géneros · 518 variables

Géneros disponibles

- ⚡ Electronic
- 🔬 Experimental
- 👤 Folk
- 🎧 Hip-Hop
- 🎸 Instrumental
- 🌐 International
- 🎵 Pop
- 🎸 Rock

Dataset: [FMA Small](#)

Audio cargado: primeros 30 s · sr=22050

> ⓘ ¿Cómo funciona?

Fork 

Clasificador de Géneros Musicales

Sube una canción · Compara 4 modelos de IA · Explora el espacio acústico

Arrastra tu canción aquí (.mp3, .wav)

 Upload 200MB per file · MP3, WAV



Sube un archivo de audio para comenzar el análisis

Formatos admitidos: MP3 · WAV · Máximo 30 segundos analizados